

Paradigma MapReduce

Semana 3

Orígenes

Contexto histórico

El paper de Google (2004)

- En 2004, Jeff Dean y Sanjay Ghemawat publicaron "**MapReduce: Simplified Data Processing on Large Clusters**"
- Google necesitaba procesar **terabytes de datos** del índice web con miles de máquinas
- El modelo abstraía la complejidad del cómputo distribuido, permitiendo a ingenieros sin experiencia en sistemas distribuidos escribir programas paralelos.

El problema a escala web

- Procesar **petabytes** de datos con hardware **commodity** (barato, no confiable)
- Las supercomputadoras son costosas y no escalan linealmente
- La solución: distribuir el trabajo en miles de nodos estándar
- Paradigma "**scale out**" (más nodos) vs. "**scale up**" (hardware más potente)

Programación distribuida

Calcular el promedio histórico de la ULima

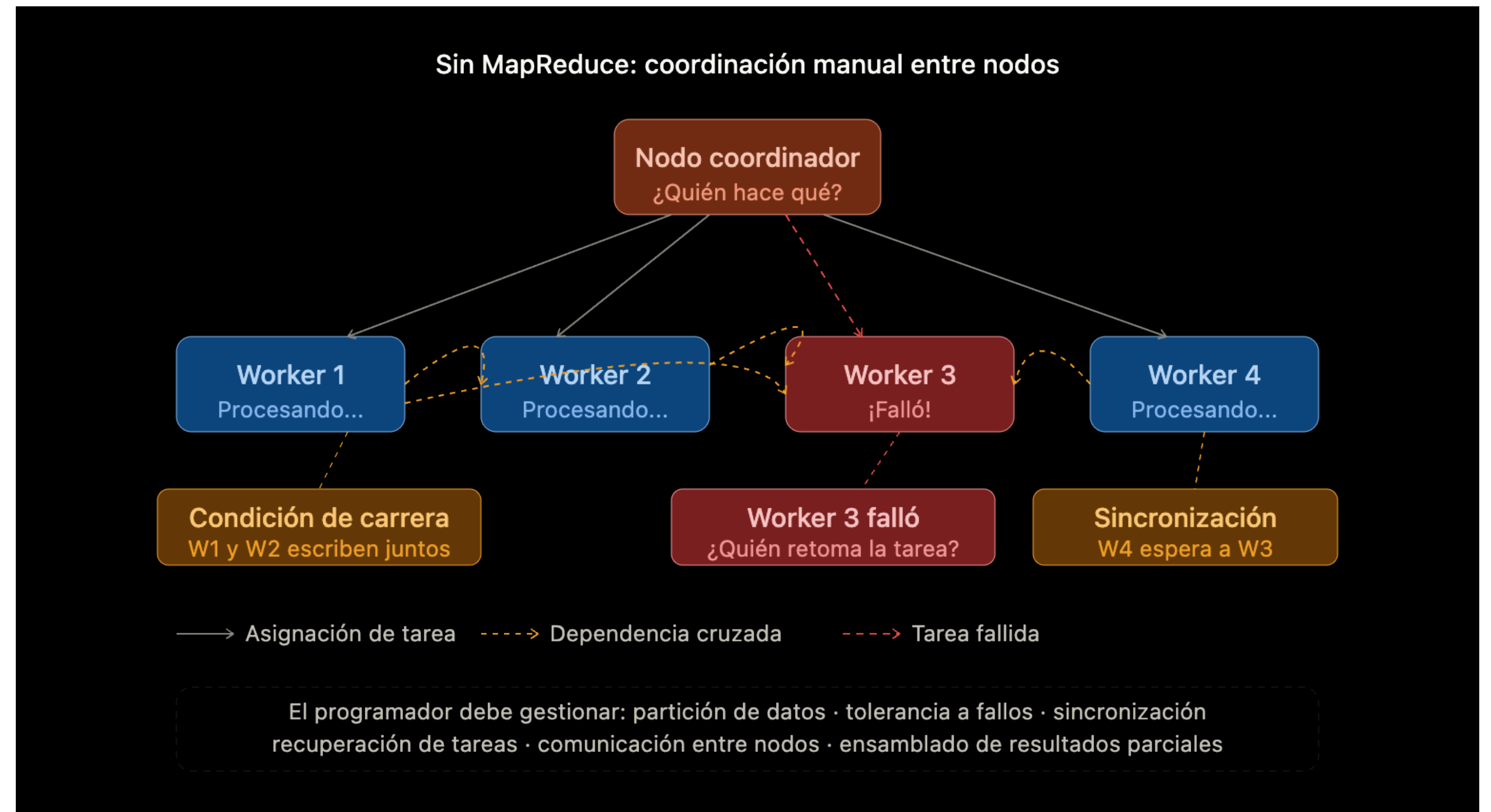
10
15
13
16
06
12
15
19
...
14

¿Cómo lo haríamos de forma distribuida?

Programación distribuida

Calcular el promedio histórico de la ULima

10
15
13
16
06
12
15
19
...
14



Principios de diseño

Principio	Descripción
Localidad de datos	Mover el cómputo al dato, no el dato al cómputo
Tolerancia a fallos	Fallos de nodos son la norma, no la excepción
Simplicidad	El programador solo define Map y Reduce; el framework hace el resto

Cornell Center for Advanced Computing. (2012). MapReduce [Training material].
<https://www.cac.cornell.edu/education/training/ParallelMay2012/MapReduce.pdf>

Modelo MapReduce

Paradigma Funcional

Pares *key-value*

```
Map(k1, v1)      → list(k2, v2)
Reduce(k2, list(v2)) → list(v3)
```

- **Map:** función **stateless** que transforma cada registro en pares `(clave, valor)`
- **Reduce:** agrega todos los valores con la misma clave
- Inspirado en lenguajes funcionales (Lisp)
- El framework gestiona automáticamente la distribución y comunicación entre nodos

Fases de Ejecución



- **Input Splits:** HDFS divide los datos en bloques (128 MB por defecto)
- **Map:** cada mapper procesa un split de forma independiente
- **Shuffle & Sort:** reagrupa pares por clave entre nodos de la red
- **Reduce:** agrega los valores agrupados por clave
- **Output:** escritura del resultado final en HDFS

Ejemplo

- Se quiere calcular el promedio de edad por provincia del Perú.



Nombre	Provincia	Edad
Ana G.	Lima	28
Carlos R.	Cusco	35
Lucía M.	Lima	42
Pedro Q.	Arequipa	50

Input Splits

Entrada: registro de personas por provincia

El dataset está almacenado en HDFS y se divide en 3 bloques. Cada bloque va a un mapper diferente.

Bloque 1 → Mapper 1

Nombre	Provincia	Edad
Ana G.	Lima	28
Carlos R.	Cusco	35
Lucía M.	Lima	42
Pedro Q.	Arequipa	50

Bloque 2 → Mapper 2

Nombre	Provincia	Edad
Rosa H.	Cusco	22
Miguel T.	Piura	60
Sofía C.	Lima	31
Jorge L.	Arequipa	45

Bloque 3 → Mapper 3

Nombre	Provincia	Edad
Elena V.	Piura	38
Luis B.	Cusco	47
María F.	Lima	55
Hector S.	Arequipa	29

12 personas en total distribuidas en 4 provincias. Queremos saber la edad promedio en cada una.

Map

Fase Map: emitir (provincia, edad)

Cada mapper recorre su bloque y emite un par por cada fila. Los 3 mappers corren en paralelo.

```
def map(fila):  
    provincia = fila["provincia"]  
    edad = fila["edad"]  
    emit(provincia, edad)
```

Mapper 1 · Bloque 1

("Lima", 28) ("Cusco", 35)
("Lima", 42)
("Arequipa", 50)

Mapper 2 · Bloque 2

("Cusco", 22) ("Piura", 60)
("Lima", 31)
("Arequipa", 45)

Mapper 3 · Bloque 3

("Piura", 38) ("Cusco", 47)
("Lima", 55)
("Arequipa", 29)

Cada mapper emite exactamente 4 pares — uno por persona. El total: 12 pares que el framework ahora debe reorganizar.

Shuffle & Sort

Shuffle & Sort: agrupar edades por provincia

El framework recoge todos los pares de los 3 mappers y los agrupa bajo la misma clave. Automático, sin código del programador.

Arequipa

edades → [50, 45, 29] · de mappers 1, 2 y 3

Cusco

edades → [35, 22, 47] · de mappers 1, 2 y 3

Lima

edades → [28, 42, 31, 55] · de mappers 1, 2 y 3

Piura

edades → [60, 38] · de mappers 2 y 3

Notar que Lima tiene 4 edades (apareció en los 3 bloques), Piura solo 2 (no estaba en el bloque 1). El shuffle transfiere datos entre nodos automáticamente.

Reduce

Fase Reduce: calcular el promedio

Cada reducer recibe una provincia con su lista de edades y calcula la media. Corren en paralelo.

```
def reduce(provincia, lista_edades):  
    promedio = sum(lista_edades) / len(lista_edades)  
    emit(provincia, round(promedio, 1))
```

Reducer A

Arequipa $\text{sum}([50, 45, 29]) = 124 / 3 = 41.3$

Cusco $\text{sum}([35, 22, 47]) = 104 / 3 = 34.7$

Reducer B

Lima $\text{sum}([28, 42, 31, 55]) = 156 / 4 = 39.0$

Piura $\text{sum}([60, 38]) = 98 / 2 = 49.0$

Solo se necesitan 2 reducers para 4 provincias. El framework decide cuántos usar según los recursos disponibles.

Output

Resultado final: edad promedio por provincia

Los resultados se escriben en HDFS y están listos para consulta o siguiente etapa del pipeline.

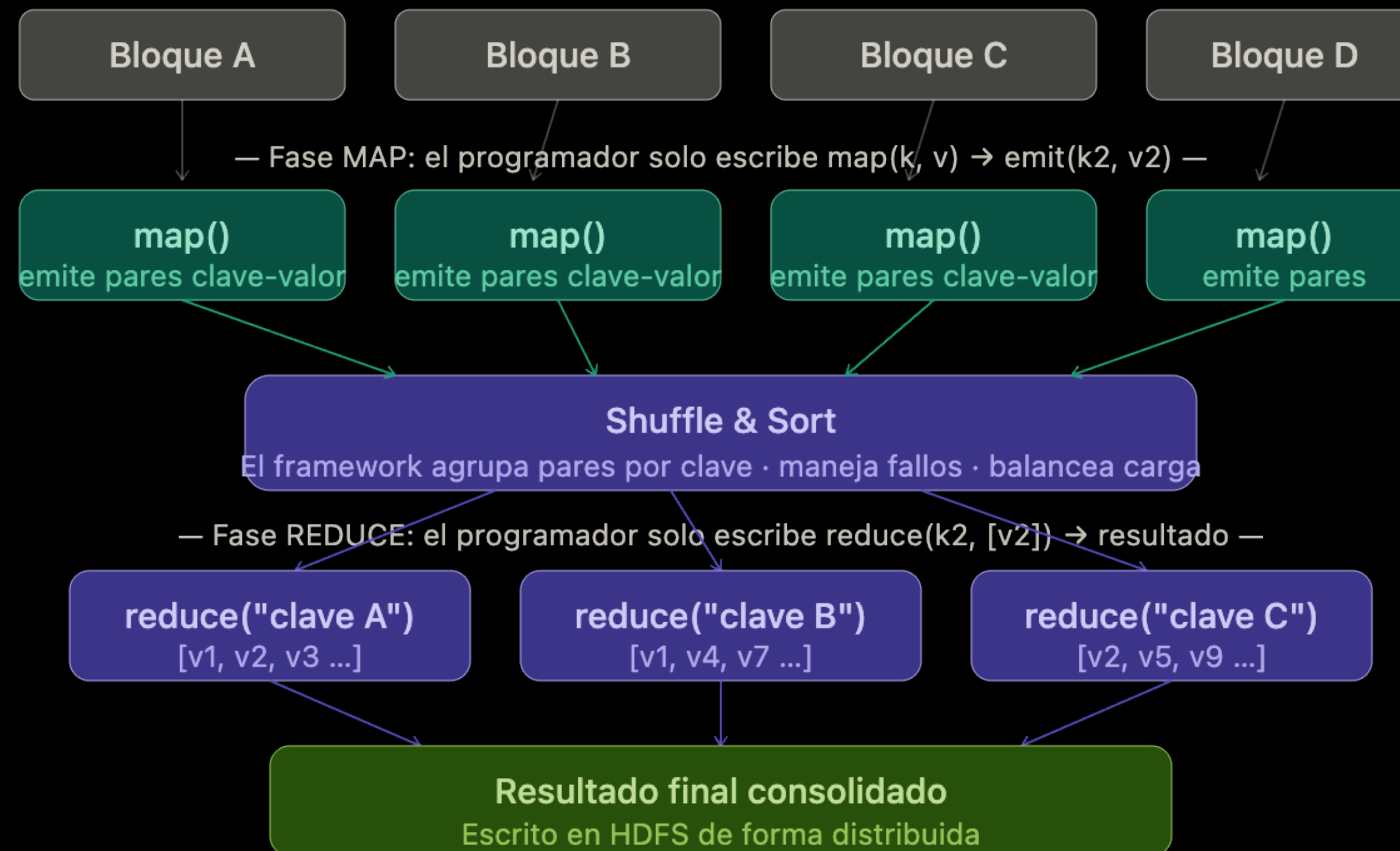
Provincia	Personas	Edades	Promedio de edad
Arequipa	3	50, 45, 29	41.3 años
Cusco	3	35, 22, 47	34.7 años
Lima	4	28, 42, 31, 55	39.0 años
Piura	2	60, 38	49.0 años

En producción real: el mismo código `map()` y `reduce()` procesaría millones de registros del RENIEC distribuidos en cientos de nodos — sin cambiar una sola línea.

Ejecución distribuida más sencilla

Con MapReduce: el framework gestiona la complejidad

Datos de entrada (ej. archivos de texto en HDFS)



El framework oculta: tolerancia a fallos · reasignación de tareas caídas · transferencia de datos entre nodos · ordenamiento distribuido · balanceo de carga automático

Referencias

Analytics Vidhya. (2022). *Apache Spark vs. Hadoop MapReduce: Top 7 differences*. <https://www.analyticsvidhya.com/blog/2022/06/apache-spark-vs-hadoop-mapreduce-top-7-differences/>

Anomaly AI. (2026). *Big data analytics tools: Hadoop, Spark, BigQuery, Snowflake*. <https://www.findanomaly.ai/big-data-analytics-tools-hadoop-spark-bigquery-snowflake>

Apache Hadoop. (n.d.). *MapReduce tutorial*. <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>

Cornell Center for Advanced Computing. (2012). *MapReduce* [Training material]. <https://www.cac.cornell.edu/education/training/ParallelMay2012/MapReduce.pdf>

Dean, J., & Ghemawat, S. (2004). MapReduce: Simplified data processing on large clusters. In *OSDI '04: Sixth Symposium on Operating System Design and Implementation*. USENIX.

Dremio. (n.d.). *MapReduce programming model*. <https://www.dremio.com/wiki/mapreduce-programming-model/>

Edureka. (n.d.). *MapReduce tutorial: Understanding MapReduce in Hadoop*. <https://www.edureka.co/blog/mapreduce-tutorial/>